

# Modularity Maximization: An Experimental Survey

Adil Chhabra  
adil.chhabra@informatik.uni-  
heidelberg.de  
Heidelberg University  
Heidelberg, Germany

Kenneth Langedal\*  
kenneth.langedal@informatik.uni-  
heidelberg.de  
Heidelberg University  
Heidelberg, Germany

Christian Schulz  
christian.schulz@informatik.uni-  
heidelberg.de  
Heidelberg University  
Heidelberg, Germany

## Abstract

Community detection methods commonly optimize modularity, an objective function designed to maximize intra-community connectivity while penalizing inter-community connectivity relative to random expectation. Over the past two decades, a wide range of algorithms have been proposed to optimize modularity, spanning classical heuristic and exact methods as well as more recent machine learning-based approaches that incorporate node attributes. Yet, survey literature and experimental evaluations remain fragmented, often confined to narrow subsets of algorithms or a single paradigm, limiting insight into relative strengths and trade-offs in practice. In this work, we present the first systematic experimental survey of modularity-based graph clustering algorithms that jointly evaluates classical and learning-based approaches. We study 17 representative algorithms spanning greedy, multilevel, metaheuristic, streaming, exact, and neural paradigms, and organize them within a unified taxonomy grounded in their algorithmic design principles. Through a consistent experimental evaluation across multiple datasets and evaluation criteria, we analyze empirical trade-offs in solution quality and scalability, providing comparative insight rather than identifying a single best method. We additionally contribute a unified, open-source framework for the implementation of all 17 evaluated clustering algorithms, supporting reproducibility and future benchmarking.

## CCS Concepts

• **Theory of computation** → **Graph algorithms analysis**; • **Mathematics of computing** → **Graph theory**; • **Computing methodologies** → **Cluster analysis**.

## Keywords

Community detection, Graph clustering, Modularity optimization, Experimental evaluation

## ACM Reference Format:

Adil Chhabra, Kenneth Langedal, and Christian Schulz. 2018. Modularity Maximization: An Experimental Survey. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email*

\*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

(Conference acronym 'XX). ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 Introduction

Networks are useful in modeling complex systems across the natural sciences, social sciences, and engineering. Such systems are naturally represented as graphs, where vertices denote entities and edges encode interactions between them. Prominent examples include online social networks, communication infrastructures, biological interaction networks, financial systems, and information networks. A common structural property of many real-world graphs is the presence of *community structure*, where vertices organize into groups that are more densely connected internally than externally.

The task of identifying these groups, commonly referred to as *graph clustering* or *community detection*, is central to network analysis. Community detection supports a wide range of applications, including the analysis of social and political behavior, fraud detection [1], protein interaction analysis and gene expression [10, 38], program optimization [13, 22], and epidemic modeling [24]. Beyond application-specific insights, community structure offers a lens into the organization of networks, highlighting cohesive substructures, boundary vertices, and inter-community connectivity patterns. Despite extensive research, it is well understood that no single clustering method dominates across all network types, motivating continued investigation into algorithmic design and comparative evaluation.

Most community-detection algorithms model clustering as an optimization problem over partitions of the vertex set, assigning a quality score to each candidate clustering. The goal is to identify a partition that optimizes this score, often under stringent computational constraints. For many commonly used objectives, finding an optimal clustering is NP-hard, and practical methods therefore rely on heuristic or approximate algorithms.

Among the many objectives proposed for graph clustering, *modularity*, introduced by Girvan and Newman [18], remains one of the most widely adopted. Modularity quantifies the density of edges within clusters compared to those between clusters, while accounting for random chance, with the goal of maximizing densely connected communities while penalizing excessive inter-community connectivity [18, 25]. Optimizing modularity is strongly NP-complete, and consequently a wide variety of heuristic algorithms have been proposed. Given the common use of modularity in the literature, this survey focuses primarily on modularity-based approaches, though we include a small number of widely used non-modularity community-detection baselines. While alternative paradigms, such as spectral clustering or probabilistic generative models, are important in the broader literature, modularity provides a common

objective that enables meaningful comparison across different algorithmic strategies.

Within modularity-based community detection, two broad classes of algorithms are identified. *Classical* approaches rely on discrete optimization heuristics that operate on graph topology. Their clustering decisions are driven solely by graph structure and a well-defined objective function, yielding transparent optimization criteria and interpretable outcomes. These methods include greedy merging, local vertex-move strategies, multilevel coarsening, metaheuristics, streaming algorithms, and exact formulations. *Learning-based* approaches integrate modularity into deep learning pipelines, most commonly through graph neural networks. These methods are motivated by the ability to incorporate node attributes, embeddings, or other side information alongside graph topology, through feature representations. This increased expressiveness introduces different trade-offs in terms of interpretability, training complexity, and computational cost.

Several challenges continue to shape the design and evaluation of clustering algorithms, driven by the scale, complexity and frequently-updating content of real-world graphs. Clustering algorithms must balance high-solution quality in contexts where data may be noisy or ground-truth communities unknown, with scalability in memory usage and runtime. Additionally, many algorithms are sensitive to pre-set parameters, such as assumed cluster counts, which complicates deployment in practice. There remains a need to systemally address how existing clustering algorithms balance trade-offs given these challenges.

Although numerous community-detection algorithms and surveys exist, there is currently no comprehensive experimental study that evaluates classical and learning-based modularity optimization methods together under a unified framework. There are limited claims about their relative strengths in the literature, and those that exist vary, particularly regarding scalability. This paper addresses this gap by providing a systematic experimental survey of modularity-based graph clustering algorithms across both classical and learning-based paradigms. Rather than advocating a single best method, this survey aims to clarify the landscape of modularity-based clustering by highlighting the strengths, limitations, and trade-offs of different algorithmic approaches across diverse settings.

## Contributions

In this study, we contribute the following:

- A taxonomy of modularity-based clustering algorithms, organizing methods by algorithmic paradigm and optimization strategy.
- A systematic review of 18 representative algorithms, covering greedy, multilevel, metaheuristic, streaming, exact, and learning-based approaches.
- The first large-scale experimental comparison of classical and learning-based modularity optimization methods, analyzing their relative performance, scalability, and trade-offs.
- A unified open-source experimental framework, including the adaptation of all evaluated algorithms behind a common interface, to facilitate future benchmarking and reproducibility in graph clustering research.

## 2 Preliminary

We use the term *algorithm* to denote a concrete computational procedure. Except where explicitly stated, the algorithms considered here are heuristics for modularity maximization and do not guarantee optimality.

**Graphs and clusterings.** An (*attributed*) graph is defined as  $G = (V, E, X)$ , where  $V = \{v_1, v_2, \dots, v_n\}$  is a set of  $n$  vertices,  $E \subseteq V \times V$  is the set of undirected edges, and  $X = \{x_1, x_2, \dots, x_n\}$  denotes node attributes, with each  $x_i \in \mathbb{R}^f$  an attribute vector associated with vertex  $v_i$ , and  $f$  the attribute dimension. We write  $n := |V|$  and  $m := |E|$  for the number of vertices and edges, respectively. For a vertex  $u \in V$ , its *neighborhood* is  $N(u) := \{v \in V : \{u, v\} \in E\}$ , and its *degree* is  $\deg(u) := |N(u)|$ . Let  $A \in \{0, 1\}^{n \times n}$  denote the adjacency matrix of  $G$ , where  $A_{ij} = 1$  if  $\{v_i, v_j\} \in E$  and  $A_{ij} = 0$  otherwise. Let  $\mathbf{d} \in \mathbb{R}^n$  denote the degree vector with  $\mathbf{d}_i = \deg(v_i)$ .

A *clustering*  $C$  of  $G$  is a partition of the vertex set  $V$  into  $k$  disjoint blocks  $V_1, \dots, V_k$  such that  $V_1 \cup \dots \cup V_k = V$ . Classical modularity-based algorithms operate solely on the graph structure  $(V, E)$ , while learning-based methods additionally exploit node attributes  $X$  when available. In particular, learning-based approaches often aim to discover communities that are coherent both in terms of graph connectivity and attribute similarity.

**Modularity.** The *modularity* score of a clustering is the fraction of edges that are within one cluster minus the expected fraction if the edges were distributed at random. Let  $L_i := |\{\{u, v\} \in E : u, v \in V_i\}|$  denote the number of edges internal to block  $V_i$ . The modularity of  $C$  is defined as

$$Q(C) := \sum_{i=1}^k \left( \frac{L_i}{m} - \left( \sum_{u \in V_i} \frac{\deg(u)}{2m} \right)^2 \right)$$

Equivalently, modularity can be expressed in spectral form. Let  $C \in \{0, 1\}^{n \times k}$  denote the (hard) cluster assignment matrix, where  $C_{ip} = 1$  if vertex  $v_i$  belongs to cluster  $p$  and  $C_{ip} = 0$  otherwise. Define the *modularity matrix*

$$B := A - \frac{\mathbf{d}\mathbf{d}^\top}{2m}.$$

Then modularity can be written as

$$Q(C) = \frac{1}{2m} \text{Tr}(C^\top B C).$$

This spectral formulation serves as the foundation for many continuous and learning-based approaches.

**Graph neural networks.** Graph neural networks (GNNs) are a class of neural architectures that compute vector representations for vertices in an attributed graph by iteratively aggregating information from local neighborhoods. Given a graph  $G = (V, E, X)$ , a GNN updates each vertex representation as a function of its own attributes and the representations of neighboring vertices, thereby encoding local graph structure and, through multiple layers, higher-order connectivity patterns.

In the context of community detection, GNNs are used to map vertices into a representation space in which structurally or semantically related vertices are close. Learning-based clustering methods then infer communities indirectly by optimizing objectives such

as continuous or differentiable relaxations of modularity, mutual-information maximization, or contrastive criteria defined over these representations or over derived *soft cluster assignments*, which encode the strength of association between nodes and communities.

### 3 Related Work

The overview and taxonomy of the clustering algorithms included in this survey follow in Section 4. First, we provide a brief overview of relevant objective functions, algorithms, and survey papers. The main motivation for this survey, and reason for focusing primarily on modularity optimization, is highlighted in Figure 1. Despite high interest from multiple scientific communities and the rapid development of new methods, comparisons between methods remain fragmented. This is a problem for both practitioners who want the best algorithm and for future researchers who need to decide which baseline algorithms to use.

While modularity has attracted significant interest, it is not without its flaws. The most well-known problem is the resolution limit [16]. In extreme cases, this can cause small communities with few edges connecting them to appear as one larger community. As a result, several algorithms provide a RESOLUTION parameter to detect smaller communities in large graphs [9, 33].

There have also been entirely new objective functions with implemented algorithms available. One such example is the *Weighted Community Cluster* (WCC), which maximizes the number of triangles within each community [28]. The SCALABLE COMMUNITY DETECTION algorithm was later developed for the WCC objective [29]. Another objective function is the *Map Equation* that measures the expected description length of a random walk [30]. Several algorithms have been developed for the map equation objective, such as INFOMAP [30], INFOFLOW [17], and the very recent learning-based NEUROMAP [8].

There have also been similar experimental surveys looking at modularity. A study by Aref et al. [4] found that modularity-optimizing heuristics rarely find optimal solutions. That claim was later disputed by Ansótegui et al. [3] who found that the VIECLUS heuristic routinely computes optimal solutions on small to medium-sized graphs.

### 4 Methods

This section briefly summarizes the clustering algorithms evaluated in our experimental survey. We organize methods primarily by *optimization paradigm*, distinguishing between *classical* combinatorial optimization methods and *learning-based* approaches with trainable parameters. Classical methods are further grouped according to their dominant optimization mechanism (e.g., greedy agglomeration, multilevel local optimization, and metaheuristics/ensembles), as well as by computational setting or guarantees (streaming algorithms and exact solvers). Learning-based methods are grouped by the role modularity plays in training, distinguishing between methods that directly optimize continuous or differentiable relaxations of modularity and those that use modularity as a guiding signal within contrastive or infomax objectives. Within each subgroup, algorithms are listed chronologically. Unless stated otherwise, methods aim to maximize modularity; we additionally include a small

**Table 1: Comparison of methods included in the experimental survey. Code indicates implementation availability (×: unavailable; ★: available but not ready for use; ★★: open source with well-defined I/O). Features indicate parallelism (P), automatic or user-specified number of clusters (A/K), learning-based methods (L), use of node features (F), and availability in portable Python packages (Py). The two bold methods are exact algorithms.**

| Method            | Year | Code | Features   |
|-------------------|------|------|------------|
| CNM [12]          | 2004 | ★★   | . A . . Py |
| WALKTRAP [27]     | 2005 | ★★   | . A . . Py |
| WT [36]           | 2007 | ★★   | . A . . Py |
| LOUVAIN [9]       | 2008 | ★★   | . A . . Py |
| CGGCI [26]        | 2012 | ×    | . A . . .  |
| VNS [2]           | 2012 | ×    | . A . . .  |
| COMBO [32]        | 2014 | ★★   | . A . . Py |
| ADVND5 [15]       | 2017 | ×    | . A . . .  |
| HOLLOCOU [20]     | 2017 | ★★   | . A . . .  |
| VIECLUS [7]       | 2018 | ★★   | P A . . .  |
| LEIDEN [33]       | 2019 | ★★   | . A . . Py |
| DMoN [34]         | 2020 | ★    | P K L F .  |
| COMM DGI [39]     | 2020 | ★    | P K L F .  |
| GNNS [31]         | 2022 | ★    | P A L . .  |
| UCoDe [23]        | 2023 | ★    | P K L F .  |
| DGCLUSTER [6]     | 2023 | ★    | P A L F .  |
| MAGI [21]         | 2024 | ★    | P K L F .  |
| <b>BAYAN</b> [5]  | 2024 | ★★   | . A . . .  |
| CLuSTRE [11]      | 2025 | ★★   | . A . . .  |
| LIM [37]          | 2025 | ★    | P A L F .  |
| <b>MaxSAT</b> [3] | 2025 | ×    | . A . . .  |

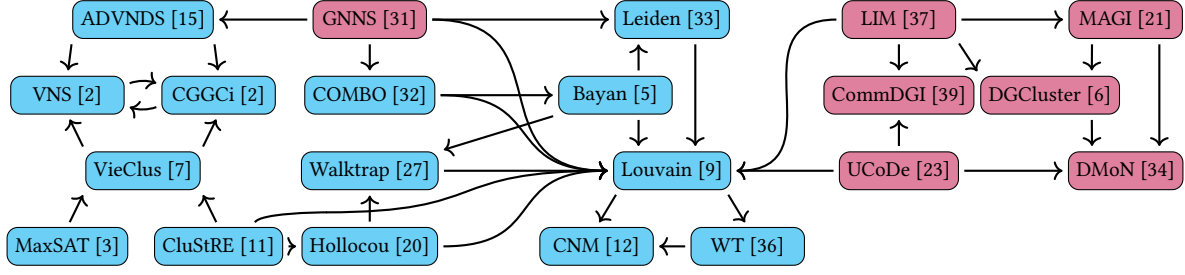
number of widely used non-modularity baselines (e.g., random-walk- and flow-based methods) for contextual comparison.

#### 4.1 Classical Modularity Optimization Methods

##### 4.1.1 Greedy agglomeration and multilevel local search.

**CNM (Clauset–Newman–Moore).** CNM is a hierarchical agglomerative algorithm that greedily maximizes modularity, starting from singleton communities (each vertex assigned to its own cluster) and repeatedly merging pairs of communities with the largest modularity gain  $\Delta Q$ . To achieve scalability, CNM computes  $\Delta Q_{c_i, c_j}$  only for community pairs  $(c_i, c_j)$  connected by at least one edge, and stores these values in balanced binary trees and max-heaps, allowing efficient updates after each merge. The algorithm produces a dendrogram that represents a hierarchical decomposition of the network into communities at all levels, and returns the clustering attaining maximum modularity. Its running time is  $O(md \log n)$ , where  $d$  is the dendrogram depth.

**WT (Wakita–Tsurumi).** WT is a refinement of CNM, motivated by the observation that CNM can produce highly unbalanced merges that degrade scalability on large-scale graphs. The algorithm preserves the hierarchical agglomerative structure of CNM, but modifies the merge-selection rule to consider both the modularity gain



**Figure 1: Previous experimental comparisons visualized.** Each directed edge indicates that the source algorithm used the destination algorithm in the experimental evaluation. The colors indicate either a learning-based method when red, or a classical optimization algorithm when blue.

$\Delta Q_{c_i, c_j}$  and a community-size ratio  $\text{ratio}(c_i, c_j)$ . This additional size-aware criterion biases the algorithm toward more balanced agglomerations.

**LOUVAIN.** LOUVAIN is a multilevel greedy algorithm that replaces CNM’s community-pair merging with local vertex moves. Instead of selecting pairs of communities  $(c_i, c_j)$  to merge based on  $\Delta Q_{c_i, c_j}$ , Louvain iteratively moves individual vertices  $u$  to neighboring communities  $c$  whenever the induced modularity gain  $\Delta Q_{u \rightarrow c}$  is positive. Once no improving vertex move exists, each community is contracted into a supernode and the process is repeated on the induced graph. Each local-moving phase runs in linear time in the number of edges, and the number of multilevel passes is typically small, yielding linear overall runtime in practice.

**LEIDEN.** LEIDEN extends LOUVAIN by adding a refinement phase that splits and restructures communities before contraction to address the possibility of disconnected or weakly connected communities produced by LOUVAIN. After the local-moving phase, each community is refined independently by starting from singleton nodes and merging vertices only with subcommunities inside the original community, provided this yields a positive modularity gain and sufficient internal connectivity. Refinement merges are non-greedy and selected probabilistically among improving options, enabling broader exploration of the search space while never decreasing modularity. As a result, Leiden guarantees that all output communities are connected and locally optimal with respect to vertex moves, while preserving the multilevel modularity-maximization framework.

#### 4.1.2 Non-modularity baselines (random-walk / flow objectives).

**WALKTRAP.** WALKTRAP is a hierarchical clustering algorithm that departs from modularity optimization and instead operates on the intuition that random walks on a graph tend to get “trapped” into densely connected regions corresponding to communities. Each vertex is characterized by the probability distribution over vertices reached by a  $t$ -step random walk starting from it, and structural similarity between vertices (and communities) is defined by comparing these distributions. A distance defined on these distributions yields a computable similarity measure between vertices and communities. Communities are then obtained by agglomerative merging of the closest pairs under this distance, producing a dendrogram similar

to CNM. The algorithm runs in  $O(mnd)$  time, where  $d$  is the height of the resulting dendrogram.

**INFOMAP.** Instead of comparing random-walk distributions as in WALKTRAP, INFOMAP seeks a clustering that minimizes the expected description length of a random walk, as formalized by the map equation. This objective is rooted in the minimum description length principle, treating community detection as a data compression problem: the optimal partition is the one that allows for the shortest possible “code” to describe the walker’s path. INFOMAP’s search mechanism is a greedy stochastic multilevel heuristic inspired by the LOUVAIN algorithm, but it incorporates additional fine-tuning and coarse-tuning steps to avoid local minima, similar to the refinements later introduced by LEIDEN. The algorithm operates in two primary phases: a two-level phase that starts with singleton nodes and iteratively moves them to neighboring modules to reduce codelength, followed by a multilevel phase that recursively groups these modules to identify hierarchical structures.

#### 4.1.3 Metaheuristics and ensemble/hybrid search.

**CGGCi-RG.** CGGCi-RG combines a randomized greedy modularity optimizer with an ensemble-based consolidation scheme known as Core Groups Graph Clustering (CGGC). CGGC extracts *core groups* of vertices that are repeatedly assigned to the same community across multiple high-quality clusterings and contracts them into supernodes to stabilize and refine the solution. The underlying Randomized Greedy (RG) algorithm follows a CNM-style hierarchical merging process, but restricts merge selection by sampling a fixed number of candidate communities and choosing the best merge among them. Multiple independent RG runs are executed to generate a diverse set of high-quality dendrograms. From these runs, core groups are identified and contracted, and modularity optimization is then repeated on the reduced graph.

**VNS.** VNS applies a *Variable Neighborhood Search* metaheuristic to modularity maximization. Starting from an initial clustering, the algorithm repeatedly selects a subproblem induced by a randomly chosen community and a growing set of neighboring communities, and perturbs the solution using one of several predefined cluster-level shaking operators (e.g., division, fusion, redistribution). Each perturbed solution is refined using a modularity-based local search (LPAm+), first within the subproblem and, upon improvement, on the full graph. If no improvement is found, the neighborhood size

is increased; otherwise it is reset, allowing systematic escape from local optima while maintaining focus on modularity optimization.

**COMBO.** COMBO is a general-purpose community-detection optimizer that performs a broader search over partitions than LOUVAIN style single-vertex moves. Starting from an initial clustering, the algorithm iteratively considers pairs of source and destination communities and computes the best redistribution of vertices between them. For each such pair, COMBO performs a Kernighan–Lin style sequence of vertex moves, starting from multiple initial configurations (including full merges and partial random transfers), and selects the configuration yielding the largest modularity gain. The best improving recombination is applied at each iteration, and the process repeats until no further improvement in modularity is possible. The algorithm runs in  $O(n^2C)$  time, where  $C$  is the number of clusters.

**VIECLUS.** VIECLUS is a memetic (evolutionary) algorithm for modularity maximization that optimizes a population of candidate clusterings rather than a single solution. In a memetic framework, each clustering is treated as an individual, and new solutions are generated by recombination and mutation: recombination exploits common structure in high-quality solutions, while mutation introduces controlled randomness to maintain diversity and escape local optima. VieCLUS initializes a diverse population using an adapted LOUVAIN method, and generates offspring via *overlay clustering*, where two vertices are grouped together only if they co-occur in both parent solutions. The resulting solution is refined using LOUVAIN or multilevel local search, while mutation is realized by splitting clusters via graph partitioning before recombination. Offspring solutions replace similar population members only if they achieve at least equal modularity, ensuring both solution quality and population diversity. After a set time limit, VIECLUS returns the best clustering found thus far.

#### 4.1.4 Streaming.

**HOLLOCOU.** HOLLOCOU is a one-pass streaming algorithm for community detection based on the observation that, under random edge orderings, intra-community edges tend to arrive earlier than inter-community edges. The algorithm starts from the singleton clustering and processes edges sequentially. For an arriving edge  $(u, v)$ , HOLLOCOU decides whether to merge the communities of  $u$  and  $v$  based on their current community volumes. If both volumes are below a fixed threshold, the node belonging to the smaller-volume community joins the other; otherwise, no merge is performed. The authors show that this local, volume-based merging rule corresponds to a greedy optimization of modularity in the streaming setting.

**CLUSTRE.** CLUSTRE follows the node-streaming model unlike the edge-streaming model of HOLLOCOU. Here, nodes arrive sequentially together with their neighborhood lists. Starting from singleton clustering, each node  $v$  is assigned to the cluster among its neighbors that maximizes the modularity gain  $\Delta Q$ , similar to a LOUVAIN local move. During streaming, a quotient graph is constructed whose nodes represent clusters and whose weighted edges summarize inter- and intra-cluster connectivity, preserving modularity exactly. This quotient graph is optionally refined using VIECLUS.

The graph may be re-streamed to perform further local improvements, iterating until modularity converges.

#### 4.1.5 Exact Solvers.

**BAYAN.** BAYAN is an exact algorithm for modularity maximization based on a branch-and-cut formulation of an integer program with binary decision variables encoding community co-membership. It enforces consistent cluster assignments using triangular constraints on node triples, which branch on whether three nodes must belong to the same community or cannot all be clustered together. Upper bounds are obtained by solving linear programming relaxations of the formulation, while lower bounds are computed from feasible clusterings produced by COMBO. Through branching on violated triangular constraints, the algorithm modifies the problem by either contracting node triples into a supernode or discouraging their co-assignment until optimality is certified or a prescribed gap is reached.

**MAXSAT.** The MAXSAT approach formulates modularity maximization as a weighted partial MaxSAT problem. Binary decision variables indicate whether pairs of vertices belong to the same community, while hard clauses enforce transitivity constraints to ensure a valid clustering. Weighted soft clauses encode pairwise contributions to modularity, such that maximizing the total satisfied weight is equivalent to maximizing modularity. To improve scalability, the formulation uses a sparse encoding that omits redundant transitivity constraints without affecting optimality. Solving the resulting MaxSAT instance yields a clustering with provably optimal modularity.

## 4.2 Learning-Based Methods

### 4.2.1 Continuous/Differentiable modularity maximization.

**DMoN (Deep Modularity Networks).** DMoN introduces an unsupervised modularity-based objective and realizes it using a GNN for graph clustering. A GNN followed by a softmax function produces soft cluster assignments, which are optimized using a fully differentiable objective whose main term corresponds to modularity in spectral form. The objective further includes a *collapse regularization* term that prevents trivial solutions in which all nodes are assigned to the same cluster.

**GNNS.** GNNS is a GNN-style method for unsupervised network clustering via modularity optimization, inspired by recurrent GNN models. Unlike discrete modularity heuristics, GNNS formulates community detection as a continuous optimization process over soft cluster assignments. Starting from random initial assignments, the algorithm iteratively updates each node's soft cluster assignment by combining its previous assignment with aggregated assignment information from neighboring nodes, weighted by the modularity matrix. Each update is followed by a normalization step to maintain valid assignment distributions, and simple activation functions are used to amplify assignments supported by positive modularity contributions while suppressing weaker ones. The procedure is repeated for multiple random initializations and parameter choices; intermediate solutions are filtered based on achieved modularity, and the final clustering is obtained by discretizing the converged soft cluster assignments.

**DGCLUSTER.** DGCLUSTER is a fully differentiable method for deep graph clustering based on graph structure and node attributes. Unlike most learning-based approaches including DMO<sub>N</sub>, DGCLUSTER does not require the number of communities to be predefined. It first computes node embeddings using a GNN that incorporates graph structure and node attributes. Similarities between node embeddings are then interpreted as soft pairwise community assignments and used to parameterize the modularity objective. This objective is optimized in a differentiable manner to train the GNN parameters, after which final community assignments are obtained by clustering the learned node embeddings using another algorithm called BIRCH. The overall complexity scales linearly with the graph size.

**LIM.** LIM (Less Is More) is a learning-based community detection method that integrates global structural information, local graph structure, and node attributes within a modularity maximization framework. It first performs community pre-detection using a LOUVAIN to extract global structural communities, which are summarized as structural community centers in a feature space. Additionally, a graph convolutional network is used to learn node representations that capture local structural information and node attributes. Soft cluster assignments are then computed by measuring similarity between node representations and the structural community centers, integrating global structure with local structural and attribute information. Finally, a modularity maximization objective defined over these soft cluster memberships is optimized to obtain the final community assignment.

#### 4.2.2 Contrastive / infomax objectives aligned with modularity.

**COMM DGI.** COMM DGI (Community Deep Graph Infomax) is a representation-learning approach to community detection that learns soft community assignments by maximizing mutual information. Building on Deep Graph Infomax, it jointly maximizes *graph mutual information* between node and graph representations and *community mutual information* between node and community (subgraph) representations to encode the graph and clustering feature of each node. A trainable clustering layer is integrated into the GNN, enabling end-to-end training for community detection. Unlike learning-based methods that directly optimize modularity (e.g., DMO<sub>N</sub>, DGCLUSTER, LIM), modularity is used here as an auxiliary objective together with mutual information, via a soft  $k$ -means-style clustering layer, to further encourage structurally coherent communities.

**UCoDE.** UCoDE (Unified Community Detection) is a GNN-based community detection method that jointly supports non-overlapping and overlapping communities within a single unsupervised framework. Like DMO<sub>N</sub> and DGCLUSTER, it represents communities through soft cluster assignments produced by a GNN, but unlike these methods it does not rely on direct modularity maximization alone. Instead, UCoDE optimizes a single contrastive objective derived from the modularity matrix, which simultaneously encourages high intra-community similarity and low inter-community similarity at the community level. This contrastive formulation allows UCoDE to recover non-overlapping communities while naturally supporting overlap, without requiring separate objectives or post-processing. Compared to combined-objective methods such

as COMM DGI, UCoDE uses a single, interpretable loss and requires minimal hyperparameter tuning.

**MAGI.** MAGI establishes a formal connection between modularity maximization and graph contrastive learning by showing that modularity naturally defines positive and negative node pairs via the sign of the modularity matrix. Specifically, node pairs with positive modularity coefficients are treated as positive samples, while pairs with non-positive coefficients are treated as negatives, yielding a contrastive objective equivalent to relaxed modularity maximization. Based on this insight, MAGI uses modularity maximization as a community-aware pretext task to train a GNN encoder with a softmax-based SimCLR contrastive loss, without relying on graph augmentations. To achieve scalability and capture high-order proximity, MAGI constructs mini-batch modularity matrices using a two-stage random-walk sampling procedure.

## 5 Evaluation Methodology

In this section, we describe the benchmarking methodology used to evaluate modularity-based graph clustering algorithms under controlled and reproducible conditions. A central contribution is the integration of 17 clustering algorithms behind a unified benchmarking interface, enabling a fair comparison across diverse implementations. In addition to enabling the experiments presented in this survey, it will also serve as an openly available resource for future work on graph clustering, simplifying the evaluation of older methods on new data or machines. This section also includes details on the datasets we used to evaluate the clustering algorithms and the computational environment.

### 5.1 Benchmark Interface

As shown in Table 1, over half of the clustering algorithms in the literature lack ready-to-use interfaces for clustering new data. Furthermore, among those with properly documented code, there are wide differences in the input graph format, output formats, and the additional information they report, such as time and memory consumption. This lack of standardization complicates fair benchmarking, as differences in preprocessing, reporting, and resource accounting can substantially affect observed performance.

To address this problem, we aim to provide an interface with a single common input and output format and consistent time and memory monitoring. The interface exposes a common set of inputs, while allowing solver-specific parameters to be provided when applicable.

$$\text{CLUSTER}(G, T_{\max}, p, k, M_{\max}, \mathcal{F})$$

Where  $G$  is the input graph,  $T_{\max}$  is the wall-clock time limit,  $p$  is the number of threads available to the algorithm,  $k$  is the number of clusters to compute,  $M_{\max}$  is the memory limit in gigabytes, and  $\mathcal{F}$  are the graph features. Parameters  $p$ ,  $k$ , and  $\mathcal{F}$  are only used by a subset of the solvers. From Table 1, parallel solvers use  $p$ , those that compute a fixed number of clusters use  $k$ , and any solver that uses graph features can optionally accept  $\mathcal{F}$ . When  $\mathcal{F}$  is not provided, but the solver expects features, a single 1.0 value is assigned to each vertex as the feature. This interface is designed to capture the most common operational requirements across clustering algorithms while enforcing consistent resource limits and output reporting.

Regarding code changes, we have tried to make as few modifications to the original source code as possible. Whenever possible, we use the authors' implementations or the most commonly used library implementation. The exact setup for each algorithm is described in our openly available GitHub repository<sup>1</sup>. We make only minimal, deliberate modifications where necessary to ensure consistent timing, memory accounting, and output quality measures based on the following rules:

- Parsing the input and writing the output to a file is not counted in the timing of the algorithm.
- For search-based algorithms that always run for the entire  $T_{\max}$  duration, the recorded computation time is defined as the point in time when the best solution was found.
- Learning-based methods that train a neural network as part of the algorithm will break the training loop at  $T_{\max}$ , and use the current model at that time to produce the final clustering.

## 5.2 Parameter Settings

As much as possible, we try to use the default configurations for all algorithms tested. In the following we detail the exact parameters chosen for each algorithm. Algorithms that are not listed do not expose any parameters.

**Parameter Selection Policy.** To reflect how algorithms are typically used in practice, we deliberately avoid any dataset-specific or instance-specific parameter tuning. Our goal is to benchmark available implementations as they are, rather than hypothetical optimal configurations that would require extensive tuning effort.

- WALKTRAP: walk length = 4.
- LOUVAIN: resolution = 1.
- HOLLOCOU:  $v_{\max} = 10\,000$ .
- VIECLUS: time limit = 1 hour.
- LEIDEN: resolution = 1,  $\beta = 0.02$ , iterations = 2.
- DMOIN: 1 hidden layer (64), 1000 epochs, lr = 0.001.
- COMMDOG: 1 hidden layer (512), 1000 epochs, lr = 0.001.
- GNNS: GNNS100 variant.
- UCODE: 2 hidden layers (256), 1000 epochs, lr = 0.001.
- DGCLUSTER: hidden layers (256, 128), 300 epochs, lr = 0.001.
- MAGI: hidden layers (512, 256), 100 epochs, lr = 0.01.
- BAYAN: Gurobi ILP solver [19], gap =  $10^{-4}$ , resolution = 1.
- CLUSTRE: light mode, modularity objective.
- LIM: 1 hidden layer (512), 1000 epochs, patience = 200, lr = 0.001.

For stochastic algorithms with an explicitly specified seed, we use the run ID as the seed for each of the five runs. While many of these algorithms include multiple configurations, we use either the configuration recommended by the authors or the default configuration provided with the implementation.

## 5.3 Dataset

We consider three datasets commonly used in the literature. The first set consists of 10 real-world graphs with known community labels; we refer to this dataset as the GROUND-TRUTH graphs. The machine learning algorithms commonly use these graphs, and the

<sup>1</sup><https://github.com/KennethLangedal/Clustering>

**Table 2: Real-world instances with known ground-truth clusters. The  $|C|$  columns indicates the number of clusters.**

| Instance         | $n$       | $m$        | $ \mathcal{F} $ | $ C $ |
|------------------|-----------|------------|-----------------|-------|
| Amazon-Computers | 13 752    | 245 861    | 767             | 10    |
| Amazon-Photo     | 7 650     | 119 081    | 745             | 8     |
| Citeseer         | 3 327     | 4 552      | 3 703           | 6     |
| Coauthor-CS      | 18 333    | 81 894     | 6 805           | 15    |
| Coauthor-Physics | 34 493    | 247 962    | 8 415           | 5     |
| Cora             | 2 708     | 5 278      | 1 433           | 7     |
| ogbn-Arxiv       | 169 343   | 1 157 799  | 128             | 40    |
| ogbn-Products    | 2 449 029 | 61 859 012 | 100             | 47    |
| Pubmed           | 19 717    | 44 324     | 500             | 3     |
| Reddit           | 232 965   | 57 307 946 | 602             | 41    |

details of the instances are shown in Table 2. Next, we consider the instances used in the 10th DIMACS implementation challenge on graph clustering; we call this dataset DIMACS graphs. This dataset is popular among the classical modularity optimization methods, and the details of these graphs are shown in Table 4 in the appendix. Finally, we use a dataset of synthetically generated graphs with node features generated using the attributed, degree-corrected stochastic block model [35] (ADC-SBM). This generator allows us to explore the algorithm's sensitivity to noise in both the graph and features.

## 5.4 Computational Environment

All algorithms are evaluated on the same machine to eliminate variability due to hardware differences and ensure fair comparison. While some methods could potentially benefit from specialized accelerators such as GPUs, we intentionally restrict all experiments to a shared CPU-based environment to maintain consistency across algorithms. We use a machine with an AMD EPYC 9754 128-core processor with 755 GB of memory, running Ubuntu 24.04.3 with Linux kernel 6.8.0-87. For algorithms with compiled software, we use GCC version 13.3.0, and for Python implementations we use Python version 3.12.3.

In order to speed up experiments, we evaluate three algorithms in parallel. For parallel algorithms, we evaluate them both sequentially and using 16 threads. If an algorithm exceeds 250 GB of memory usage on an input instance, it is killed and recorded as *memory limit exceeded* for that graph. Similarly, we assign each algorithm a time limit of 1 hour per input graph. Unlike the memory limit, we allow limited flexibility for time limits due to the structure of learning-based methods, which often train for a fixed number of iterations. Only if an algorithm overshoots the time limit by more than 50% do we kill the process and record that instance as *time limit exceeded*.

For algorithms that needs to be told the number of clusters, we either provide the exact number when ground truth labels are known, or 16 clusters otherwise, as is commonly used in the literature. Since several of the algorithms are Stochastic, we report the best result over five runs to reflect the maximum achievable solution quality under the given resource constraints, which is standard practice in modularity optimization. In addition to measuring modularity as

defined in Section 2, we also use Normalized Mutual Information (NMI) between the computed clusters and ground-truth labels and F1 scores between adjacent pairs of vertices.

**Performance Profiles.** Performance profiles are well suited for benchmarking diverse algorithms, as they summarize relative performance across instances without requiring direct aggregation of objective values [14]. A performance profile assigns each included algorithm a non-decreasing, piecewise-constant function. The y-axis represents the fraction of graphs where the objective value is better than or equal to  $\tau$  times the best observed objective function value from any algorithm. Reading a performance profile, an algorithm performs well if it achieves lower  $\tau$  values for higher fractions of instances.

## 6 Experiments

In this section, we present the results of our experimental evaluation of the 17 algorithms with openly available implementations. This section is divided into three main parts. The first part presents results for the real-world dataset shown in Table 2, and the instances from the 10th DIMACS implementation challenge shown in Table 4. The second part investigates the computational efficiency of the algorithms, including execution time and memory consumption, as well as the effect that parallel processing has on the algorithms that support it. And finally, the third part presents results for our synthetically generated graphs with varying amounts of noise applied to the graph structure and the node features.

### 6.1 State-of-the-Art Comparisons

Figure 1 shows the main results of our experimental comparison. The first thing to point out is that VIECLUS achieves the best result on *all* instances when it finishes within the time and memory constraints. There are two graphs where VIECLUS exceeded the memory limit: uk-2002 and uk-2007-05, both from the DIMACS set. This result aligns with the results presented by the authors of VIECLUS, who excluded these two graphs in particular because they were too large for a memetic algorithm [7].

For the GROUND-TRUTH dataset, many algorithms achieve very similar F1 scores. However, the MAGI method stands out as the best-performing algorithm on this dataset, achieving consistently high F1 scores and the best worst-case performance. It is worth noting that most algorithms optimize modularity, while the ground-truth clusters could have relatively low modularity. For instance, the ground-truth clusters for the Citeseer graph have a modularity score of 0.54, much lower than the highest recorded modularity by VIECLUS of 0.89. In fact, all tested algorithms compute clusterings with higher modularity scores than the ground-truth clustering on this graph. One reason for this discrepancy is how the ground-truth clusters are defined. The Citeseer graph is a citation network in which vertices are scientific publications and edges represent citations. The ground-truth clusters represent categories of each publication. As Figure 1 would indicate, there is no guarantee that a particular publication favors citations within its own category, which would have aligned perfectly with the modularity objective.

For the second DIMACS dataset, the differences are more pronounced, with VIECLUS standing out as the clearly best performing method. Following VIECLUS are the closely related algorithms

LEIDEN and LOUVAIN, both achieving modularity scores close to VIECLUS. Looking at learning-based methods, there are significant differences in how they handle graphs without vertex features. Algorithms such as UCODE, which was among the strongest learning-based algorithms on the GROUND-TRUTH dataset, struggle to produce meaningful clusterings on DIMACS graphs. On the other hand, DMOÑ and COMMDGI perform well despite being designed for graphs with node features and a known number of clusters. Unsurprisingly, GNNS performs best among the learning-based methods, being the only one designed for graphs without features.

The streaming algorithms HOLLOCOU and CLUSTRE perform similarly on the GROUND-TRUTH dataset, but are noticeably worse than the other algorithms tested in terms of F1 scores. On the DIMACS dataset, this changes, and especially the CLUSTRE algorithm outperforms several learning-based algorithms. It also gets close to matching the WALKTRAP method in terms of modularity. All while running extremely fast and with a low memory footprint.

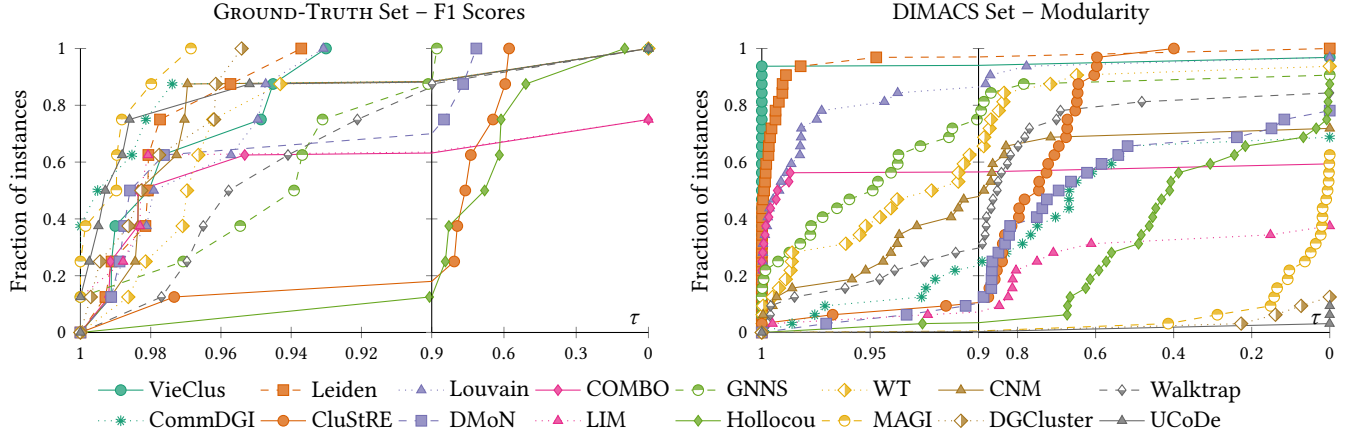
The exact algorithm BAYAN is not included in the performance profiles, as it terminated on only the eight smallest instances from the DIMACS dataset. Among these eight instances, VIECLUS found the optimal clustering for all of them, LEIDEN for 7/8, GNNS for 5/8, COMBO for 4/8, and LOUVAIN for 3/8. The remaining algorithms did not find any known optimal solutions.

### 6.2 Computational Efficiency

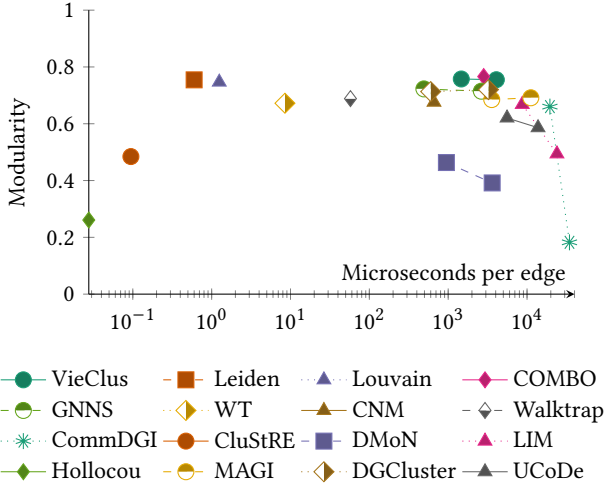
Judging computational efficiency is made difficult by the wide variety of methods. Figure 3 shows the relation between clustering quality and computational time to highlight the main tendencies of the algorithms. Reading the figure from left to right, the streaming algorithms HOLLOCOU and CLUSTRE are by far the fastest. Next are the two closely related algorithms LEIDEN and LOUVAIN, both scoring near the top in both quality and execution time, only beaten by VIECLUS in terms of solution quality. Both WT and WALKTRAP take longer and produce lower-quality solutions than LEIDEN and LOUVAIN. Unsurprisingly, most learning-based methods take a long time to produce a clustering. Still, they also achieve generally high solution quality.

In terms of parallel scalability, all the tested algorithms benefit from having access to more computing resources. The algorithms VIECLUS, GNNS, and MAGI achieve very similar solution qualities in parallel and sequential runs, but run faster in parallel. While DMOÑ, UCODE, LIM, and COMMDGI both run faster and produce higher quality solutions. The difference is especially noticeable for COMMDGI, who struggle to finish enough training iterations within the one-hour time limit sequentially.

Table 3 shows the peak memory consumption for each algorithm on the Coauthor-Physics graph. As with the running times, the differences here are also very noticeable. Both of the streaming algorithms use less than 10 KB of memory. In contrast, the learning-based methods range from 7.4 GB for DMOÑ to 31.1 GB for COMMDGI. VIECLUS, LOUVAIN, LEIDEN, and WT are all in the 50-100 MB range, while other non-learning methods also need a lot more memory, such as COMBO with 18.8 GB and WALKTRAP with 4.4 GB.



**Figure 2: Performance profiles for our first two datasets, the real-world graphs and the instances from the 10th DIMACS implementation challenge. For real-world instances, we use F1 scores to measure solution quality, and for the DIMACS instances, we use modularity. To improve readability, we split the x-axis to highlight the differences between the best algorithms near 1.**



**Figure 3: Scatter plot showing the ratio between running time and solution quality, using the geometric mean for both measures. To normalize the running time for each graph, we divide the running time by the number of edges, resulting in a measure of time per edge. Each tested parallel algorithm is represented by two points: one for the parallel version and one for the sequential version. For all the algorithms tested, the parallel versions consistently demonstrated lower running times than their sequential counterparts, meaning the leftmost point on the graph always represents the parallel version.**

### 6.3 Syntetic Graphs

To further evaluate the impact and sensitivity of graph features across clustering algorithms, we include experiments with syntactically generated graphs with vertex features. The generator we use is called the attributed, degree-corrected stochastic block model [35]

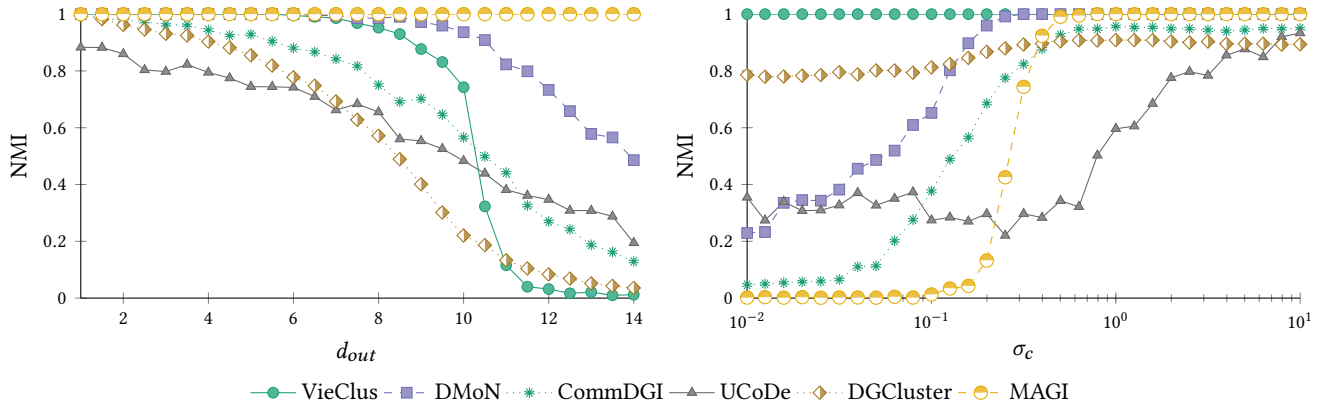
**Table 3: Memory consumption of different algorithms on the Coauthor-Physics graph. Note that this graph has  $n = 34\,493$ ,  $m = 247\,962$ , and  $|\mathcal{F}| = 8\,415$ .**

| Alg.     | Mem.    | Alg.     | Mem.    | Alg.      | Mem.    |
|----------|---------|----------|---------|-----------|---------|
| VieCLUS  | 0.1 GB  | LEIDEN   | 61.4 MB | LOUVAIN   | 61.4 MB |
| COMBO    | 18.8 GB | GNNS     | 8.1 GB  | WT        | 61.4 MB |
| CNM      | 0.4 GB  | WALKTRAP | 4.4 GB  | COMM DGI  | 31.1 GB |
| CluStRE  | 6.1 KB  | DMoN     | 7.4 GB  | LIM       | 24.5 GB |
| HOLLOCOU | 8.1 KB  | MAGI     | 15.7 GB | DGCLUSTER | 13.2 GB |
| UCoDe    | 24.2 GB |          |         |           |         |

(ADC-SBM). This generator plants  $k$  clusters in the graph, and generates random edges via a distribution that is conditional on those clusters. It does so using the two input parameters  $d$  and  $d_{out} \leq d$ , where  $d$  is the expected degree for each vertex and  $d_{out}$  is the expected number of neighbors for a node in a different cluster than its own. The ADC-SBM generator also produces node features that are correlated with the community structure. It does so using the two input parameters  $\sigma$  for intra-cluster center variance and  $\sigma_c$  for cluster center variance.

We conduct two sets of experiments with the ADC-SBM generator. First, we generate a sequence of graphs with gradually higher  $d_{out}$ , causing the graph structure to carry less information about the original  $k$  clusters. And second, we generate graphs with strong community structure as we gradually increase the cluster center variance  $\sigma_c$ , leading the graph features to carry less information about the clusters.

Figure 4 shows the results of our two experiments with the ADC-SBM graph generator. Since this experiment aims to show how feature-aware methods respond to scenarios in which either the graph or the feature signal is distorted, we evaluate only methods that accept graph features as input. For reference, we also



**Figure 4: NMI scores on graphs generated with the ADC-SBM graph generator. All graphs have  $n = 1\,000$  vertices,  $k = 4$  clusters,  $d = 20$  average degree, and  $\sigma = 1$  intra-cluster center variance. The left plot varies  $d_{out}$  while keeping  $\sigma_c = 3$ , and the right plot varies  $\sigma_c$  while keeping  $d_{out} = 2$ .**

include VIECLUS, which performed best among the pure modularity-optimizing methods. When the feature signal is strong, MAGI and DMOÑ are the best performers, and at around  $d_{out} = 11$ , all the feature-aware methods perform better than VIECLUS. It is also clear from this point of view that MAGI relies more on graph features than the graph structure, evident by the strong performance when the graph structure is distorted, and weak performance when the feature signal is distorted. DMOÑ performs well in both scenarios, demonstrating its ability to capture community structure in both the graph and the features without relying too heavily on either one. Similarly, COMMDDGI displays similar performance, but is noticeably worse than DMOÑ across the board. DGCLUSTER is somewhat opposite to MAGI, favoring the graph structure over the features. The same holds for UCODE, but to a lesser extent. We excluded LIM from this figure because it failed to compute more than a single cluster for these generated graphs, resulting in a constant zero NMI score across configurations.

## 7 Conclusion

We surveyed over twenty modularity-based graph clustering algorithms, spanning classical heuristic methods, parallel implementations, and recent learning-based approaches. When modularity maximization is the sole objective, the results are clear: VIECLUS consistently achieves the highest modularity scores, provided sufficient memory is available. LEIDEN, an improved variant of LOUVAIN, follows closely in terms of solution quality while offering substantially faster runtimes. This difference can be attributed in part to the search-based nature of VIECLUS, which requires users to explicitly specify a time budget, whereas LEIDEN employs a greedy optimization strategy.

Beyond modularity alone, different algorithms exhibit distinct strengths. For example, the streaming method CLUSTRE achieves extremely low runtimes and memory consumption, making it attractive for large-scale or resource-constrained settings. Learning-based methods, such as GNNS, demonstrate that neural approaches are approaching the performance of manually designed heuristics. However, they generally require significantly more memory and

do not yet achieve competitive modularity values. At the same time, there is growing interest in combining modularity optimization with auxiliary objectives to recover ground-truth communities better. In this setting, methods that exploit node features can be advantageous, such as MAGI that consistently achieves the best recovery of ground-truth clusters in our experiments. In contrast, approaches such as DMOÑ remain effective when it is unclear whether community structure is primarily encoded in the graph topology or in node attributes.

Finally, to facilitate future experimental research in this area, we provide a unified interface that enables direct, reproducible comparisons across a wide range of modularity-based clustering algorithms. We hope that this framework lowers the barrier for evaluating new methods against established baselines and encourages more systematic experimental studies of graph clustering algorithms.

## Acknowledgments

We acknowledge support by DFG grant SCHU 2567/5-1.

## References

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph based anomaly detection and description: a survey. *Data Min. Knowl. Discov.* 29, 3 (2015), 626–688. doi:10.1007/S10618-014-0365-Y
- [2] Daniel Aloise, Gilles Caporossi, Pierre Hansen, Leo Liberti, Sylvain Perron, and Manuel Ruiz. 2012. Modularity maximization in networks by variable neighborhood search. In *Graph Partitioning and Graph Clustering*. Contemporary Mathematics, Vol. 588. American Mathematical Society, 113–127. doi:10.1090/conm/588/11705
- [3] Carlos Ansótegui, Vaidyanathan Peruvemba Ramaswamy, Stefan Szeider, and Hai Xia. 2025. Uncovering and Verifying Optimal Community Structure in Complex Networks: A MaxSAT Approach. In *Proceedings of the 25th International Conference on Computational Science (ICCS 2025)*. Springer Nature Switzerland, 35–49. doi:10.1007/978-3-031-97629-2\_3
- [4] Samin Aref, Mahdi Mostajabdeh, and Hriday Chheda. 2023. Heuristic Modularity Maximization Algorithms for Community Detection Rarely Return an Optimal Partition or Anything Similar. In *Proceedings of the 23th International Conference on Computational Science (ICCS 2023)*. Springer Nature Switzerland, 612–626. doi:10.1007/978-3-031-36027-5\_48
- [5] Samin Aref, Mahdi Mostajabdeh, and Hriday Chheda. 2024. Bayan algorithm: Detecting communities in networks through exact and approximate optimization of modularity. *Physical Review E* 110, 4 (2024), 044315. doi:10.1103/PhysRevE.110.044315

- [6] Aritra Bhowmick, Mert Kosan, Zexi Huang, Ambuj Singh, and Sourav Medya. 2024. DGLCLUSTER: A Neural Framework for Attributed Graph Clustering via Modularity Maximization. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI 2024)*. AAAI Press, Washington, DC, USA, 11069–11077. doi:10.1609/aaai.v38i10.28983
- [7] Sonja Biedermann, Monika Henzinger, Christian Schulz, and Bernhard Schuster. 2018. Memetic Graph Clustering. In *Proceedings of the 17th International Symposium on Experimental Algorithms (SEA 2018)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 3:1–3:15. doi:10.4230/LIPIcs.SEA.2018.3
- [8] Christopher Blöcker, Chester Tan, and Ingo Scholtes. 2024. The map equation goes neural: Mapping network flows with graph neural networks. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024)*. Curran Associates, Inc., 17428–17456. doi:10.52202/079017-0554
- [9] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008. doi:10.1088/1742-5468/2008/10/P10008
- [10] Bingjing Cai, Haiying Wang, Hui Wang, and Hui Wang. 2011. Detection of functional modules from protein interaction networks with an enhanced random walk based algorithm. *Int. J. Comput. Biol. Drug Des.* 4, 3 (2011), 290–306. doi:10.1504/IJCBD.2011.041416
- [11] Adil Chhabra, Shai Dorian Peretz, and Christian Schulz. 2025. CluStRE: Streaming Graph Clustering with Multi-Stage Refinement. In *Proceedings of the 23rd International Symposium on Experimental Algorithms (SEA 2025)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 11:1–11:20. doi:10.4230/LIPIcs.SEA.2025.11
- [12] Aaron Clauset, Mark E. J. Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical Review E* 70, 6 (2004), 066111. doi:10.1103/PhysRevE.70.066111
- [13] John Demme and Simha Sethumadhavan. 2012. Approximate graph clustering for program characterization. *ACM Trans. Archit. Code Optim.* 8, 4 (2012), 21:1–21:21. doi:10.1145/2086696.2086700
- [14] Elizabeth D. Dolan and Jorge J. Moré. 2002. Benchmarking optimization software with performance profiles. *Mathematical Programming* 91, 2 (2002), 201–213. doi:10.1007/s101070100263
- [15] Dušan Džamić, Daniel Aloise, and Nenad Mladenović. 2019. Ascent-descent variable neighborhood decomposition search for community detection by modularity maximization. *Annals of Operations Research* 272, 1 (2019). doi:10.1007/s10479-017-2553-9
- [16] Santo Fortunato and Marc Barthélemy. 2007. Resolution limit in community detection. *Proceedings of the national academy of sciences* 104, 1 (2007), 36–41. doi:10.1073/pnas.0605965104
- [17] Park K. Fung. 2019. InfoFlow: A Distributed Algorithm to Detect Communities According to the Map Equation. *Big Data and Cognitive Computing* 3, 3 (2019), 42. doi:10.3390/bdcc3030042
- [18] M. Girvan and M. E. J. Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.122653799 doi:10.1073/pnas.122653799
- [19] Gurobi Optimization, LLC. 2026. Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
- [20] Alexandre Holloccou, Julien Maudet, Thomas Bonald, and Marc Lelarge. 2017. A Streaming Algorithm for Graph Clustering. arXiv preprint. doi:10.48550/arXiv.1712.04337
- [21] Yunfei Liu, Jintang Li, Yuehe Chen, Ruofan Wu, Ericbk Wang, Jing Zhou, Sheng Tian, Shuheng Shen, Xing Fu, Changhua Meng, Weiqiang Wang, and Liang Chen. 2024. Revisiting Modularity Maximization for Graph Clustering: A Contrastive Learning Perspective. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2024)*. Association for Computing Machinery, New York, NY, United States, 1968–1979. doi:10.1145/3637528.3671967
- [22] Scott McFarling. 1989. Program Optimization for Instruction Caches. In *ASPLOS-III Proceedings - Third International Conference on Architectural Support for Programming Languages and Operating Systems, Boston, Massachusetts, USA, April 3-6, 1989*. Joel S. Emer and John L. Hennessy (Eds.). ACM Press, 183–191. doi:10.1145/70082.68200
- [23] Atefeh Moradan, Andrew Draganov, Davide Mottin, and Ira Assent. 2023. UCoDe: Unified community detection with graph convolutional networks. *Machine Learning* 112, 12 (2023), 5057–5080. doi:10.1007/s10994-023-06402-0
- [24] M. E. J. Newman. 2003. Properties of highly clustered networks. *Phys. Rev. E* 68 (Aug 2003), 026121. Issue 2. doi:10.1103/PhysRevE.68.026121
- [25] M. E. J. Newman. 2006. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences* 103, 23 (2006), 8577–8582. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.0601602103 doi:10.1073/pnas.0601602103
- [26] Michael Ovelgönne and Andreas Geyer-Schulz. 2012. An ensemble learning strategy for graph clustering. In *Graph Partitioning and Graph Clustering*. Contemporary Mathematics, Vol. 588. American Mathematical Society, 187–205. doi:10.1090/conm/588/11705
- [27] Pascal Pons and Matthieu Latapy. 2005. Computing Communities in Large Networks Using Random Walks. In *Proceedings of the 20th International Symposium on Computer and Information Sciences (ISCIS 2005)*. Springer Berlin Heidelberg, 284–293. doi:10.1007/11569596\_31
- [28] Arnau Prat-Pérez, David Dominguez-Sal, Josep M. Brunat, and Josep-Lluís Larriba-Pey. 2012. Shaping communities out of triangles. In *Proceedings of the 21st ACM international conference on Information and knowledge management*. Association for Computing Machinery, 1677–1681. doi:10.1145/2396761.2398496
- [29] Arnau Prat-Pérez, David Dominguez-Sal, and Josep-Lluís Larriba-Pey. 2014. High quality, scalable and parallel community detection for large real graphs. In *Proceedings of the 23rd International Conference on World Wide Web (WWW 2014)*. Association for Computing Machinery, 225–236. doi:10.1145/2566486.2568010
- [30] Martin Rosvall and Carl T. Bergstrom. 2008. Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences* 105, 4 (2008), 1118–1123. doi:10.1073/pnas.0706851105
- [31] Stanislav Sobolevsky and Alexander Belyi. 2022. Graph neural network inspired algorithm for unsupervised network community detection. *Applied Network Science* 7, 1 (2022), 63. doi:10.1007/s41109-022-00500-z
- [32] Stanislav Sobolevsky, Riccardo Campari, Alexander Belyi, and Carlo Ratti. 2014. General optimization technique for high-quality community detection in complex networks. *Physical Review E* 90, 1 (2014), 012811. doi:10.1103/PhysRevE.90.012811
- [33] Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* 9, 1 (2019), 5233. doi:10.1038/s41598-019-41695-z
- [34] Anton Tsitsulin, John Palowitch, Bryan Perozzi, and Emmanuel Müller. 2023. Graph Clustering with Graph Neural Networks. *Journal of Machine Learning Research* 24, 127 (2023), 1–21. <http://jmlr.org/papers/v24/20-998.html>
- [35] Anton Tsitsulin, Benedek Rozemberczki, John Palowitch, and Bryan Perozzi. 2022. Synthetic Graph Generation to Benchmark Graph Learning. arXiv preprint. doi:10.48550/arXiv.2204.01376
- [36] Ken Wakita and Toshiyuki Tsurumi. 2007. Finding community structure in mega-scale social networks: [extended abstract]. In *Proceedings of the 16th International Conference on World Wide Web (WWW 2007)*. Association for Computing Machinery, 1275–1276. doi:10.1145/1242572.1242805
- [37] Hong Wang, Yinglong Zhang, Zhangqi Zhao, Zhicong Cai, Xuewen Xia, and Xing Xu. 2025. Simple yet effective heuristic community detection with graph convolution network. *Scientific Reports* 15, 1 (2025), 39249. doi:10.1038/s41598-025-22860-z
- [38] Ying Xu, Victor Olman, and Dong Xu. 2002. Clustering gene expression data using a graph-theoretic approach: an application of minimum spanning trees. *Bioinform.* 18, 4 (2002), 536–545. doi:10.1093/BIOINFORMATICS/18.4.536
- [39] Tianqi Zhang, Yun Xiong, Jiawei Zhang, Yao Zhang, Yizhu Jiao, and Yangyong Zhu. 2020. CommDGI: Community Detection Oriented Deep Graph Infomax. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM 2020)*. Association for Computing Machinery, 1843–1852. doi:10.1145/3340531.3412042

**Table 4: Graph from the 10th DIMACS implementation challenge. There are no features or ground truth clusters for these graphs.**

| Instance               | <i>n</i>    | <i>m</i>      |
|------------------------|-------------|---------------|
| jazz                   | 198         | 2 742         |
| celegans-metabolic     | 453         | 2 025         |
| email                  | 1 133       | 5 451         |
| PGPgiantcompo          | 10 680      | 24 316        |
| adjnoun                | 112         | 425           |
| as-22july06            | 22 963      | 48 436        |
| astro-ph               | 16 706      | 121 251       |
| celegansneural         | 297         | 2 148         |
| cond-mat               | 16 726      | 47 594        |
| cond-mat-2003          | 31 163      | 120 029       |
| cond-mat-2005          | 40 421      | 175 691       |
| dolphins               | 62          | 159           |
| football               | 115         | 613           |
| hep-th                 | 8 361       | 15 751        |
| karate                 | 34          | 78            |
| lesmis                 | 77          | 254           |
| netscience             | 1 589       | 2 742         |
| polblogs               | 1 490       | 16 715        |
| polbooks               | 105         | 441           |
| power                  | 4 941       | 6 594         |
| cnr-2000               | 325 557     | 2 738 969     |
| eu-2005                | 862 664     | 16 138 468    |
| in-2004                | 1 382 908   | 13 591 473    |
| uk-2002                | 18 520 486  | 261 787 258   |
| uk-2007-05             | 105 896 555 | 3 301 876 564 |
| road-central           | 14 081 816  | 16 933 413    |
| road-usa               | 23 947 347  | 28 854 312    |
| chesapeake             | 39          | 170           |
| caidaRouterLevel       | 192 244     | 609 066       |
| preferentialAttachment | 100 000     | 499 985       |
| smallworld             | 100 000     | 499 998       |
| G-n-pin-pout           | 100 000     | 501 198       |

**A Instances**

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009